K Nearest Neighbor, Decision Trees, and Ensembles

Fraida Fund

Contents

In this lecture . . . 2
Parametric vs. non-parametric models . . . 3
So far... . . . 3
Parametric models . . . 3
Non-parametric models . . . 3
K nearest neighbors . . . 3
1-NN . . . 4
1NN - approximation as a step function . . . 5
1NN - decision boundaries . . . 5
K nearest neighbors . . . 6
KNN for classification . . . 6
KNN for regression . . . 6
What value of K? Illustration (1NN) . . . 7
What value of K? Illustration (2NN) . . . 7
What value of K? Illustration (3NN) . . . 8
What value of K? Illustration (9NN) . . . 8
What value of K? (1) . . . 8
What value of K? (2) . . . 9
What distance measure? (1) . . . 10
Distance measure - standardization (1) . . . 10
Distance measure - standardization (2) . . . 10
Distance measure - equal weighted features . . . 11
Nearest neighbor runtime . . . 11
Summary of NN method . . . 12
The good and the bad (1) . . . 12
The good and the bad (2) . . . 12
The good and the bad (3) . . . 12
Flexible decisions with cheap prediction? . . . 13
Decision trees . . . 13
Tree terminology . . . 13
Note on notation . . . 13
Stratification of feature space (1) . . . 14
Stratification of feature space (2) . . . 14
Stratification of feature space - illustration . . . 15
Approximation using steps . . . 15
Training a decision tree . . . 16
Basic idea . . . 16
Recursive binary splitting steps . . . 16
Recursive binary splitting . . . 16
Loss function for regression tree . . . 16
Loss function for classification tree . . . 17

Measures of node impurity . . . 17
Comparison - measures of node impurity . . . 17
Conditional entropy . . . 18
Information gain . . . 18
Example: should I play tennis? (1) . . . 18
Example: should I play tennis? (2) . . . 18
Example: should I play tennis? (3) . . . 18
Example: should I play tennis? (4) . . . 19
Example: should I play tennis? (5) . . . 19
Variations . . . 19
Bias and variance . . . 20
Managing tree depth . . . 20
Stopping criteria . . . 20
Pruning . . . 20
Weakest link pruning (1) . . . 21
Weakest link pruning (2) . . . 21
Cost complexity pruning . . . 22
Runtime . . . 22
Inference time . . . 22
Summary - so far . . . 23
The good and the bad (1) . . . 23
The good and the bad (2) . . . 23
Ensembles . . . 24
Recap: decision trees . . . 24
Ensemble methods - the idea . . . 24
Ensemble methods - types (1) . . . 24
Ensemble methods - types (2) . . . 24
Bagging . . . 24
Bagging - background . . . 24
Bootstrapping . . . 24
Bootstrap aggregation . . . 24
Bagging trees . . . 25
Correlated trees . . . 25
Random forests . . . 25
Bagged trees illustration . . . 25
A note on computation . . . 25
Boosting . . . 26
Boosting - training . . . 26
AdaBoost (Adaptive Boosting) . . . 26
AdaBoost algorithm . . . 26
AdaBoost algorithm (inner loop) . . . 26
AdaBoost algorithm (final step) . . . 26
Gradient Boosting . . . 27
Recap . . . 27
Summary of (selected) ensemble methods . . . 27

In this lecture

- Parametric vs. non-parametric models
- Nearest neighbor
- Decision trees
- Ensembles of trees: bagging, random forests, boosting

Parametric vs. non-parametric models

So far...

All of our models have looked like

$$\hat{y} = f(x, w) = w_0 + w_1 x_1 + \cdots + w_d x_d$$

A model class is more flexible if $f(x, w)$ can represent more possible functions.

- Some possibilities for $f(x, w) = w_0 + w_1 x_1$
- More possibilities for $f(x, w) = w_0 + w_1 x_1 + w_2 x_2$
- Even more possibilities for $f(x, w) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$

But until now, we had to "know" how to add flexibility - for example, by adding interaction terms or other basis functions.

A way to get more flexible models is with a **non-parametric** approach, where we don't a priori impose the functional form or a fixed number of parameters that the model should learn.

Parametric models

- A particular model class is assumed (e.g. linear)
- Number of parameters fixed in advance

Note: even if you use K-fold CV to try different candidate models with different number of parameters, you are still defining each candidate model in advance.

Non-parametric models

- Minimal assumptions about model class
- Model structure determined by data

K nearest neighbors

- A kind of non-parametric model.
- Basic idea: Find labeled samples that are "similar" to the new sample, and use their labels to make prediction for the new sample.

We previously spoke about the inductive bias of the linear models - we assumed that the target variable can be modeled as a linear combination of features or transformed versions of features. What is the inductive bias here?

1-NN

- Given training data $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$
- And a new sample $\mathbf{x}_0$
- Find the sample in the training data $\mathbf{x}_{i'}$ with the least distance to $\mathbf{x}_0$.

$$i' = \underset{i=1,\ldots,N}{\text{argmin}}\ d(\mathbf{x}_i, \mathbf{x}_0)$$

- Let $\hat{y}_0 = y_{i'}$



Figure 1: The nearest neighbor to the orange test point is the one that has the smallest distance in the feature space.

1NN - approximation as a step function

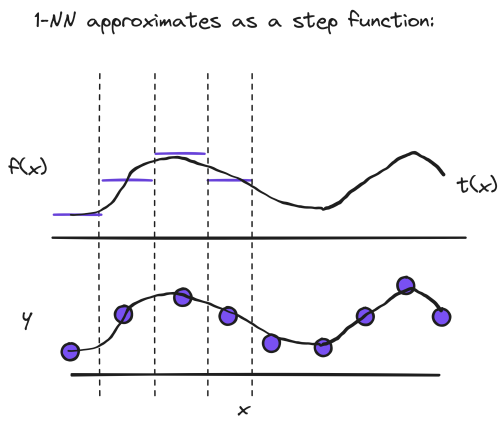


Figure 2: The nearest neighbor model approximates the true function using steps.

1NN - decision boundaries

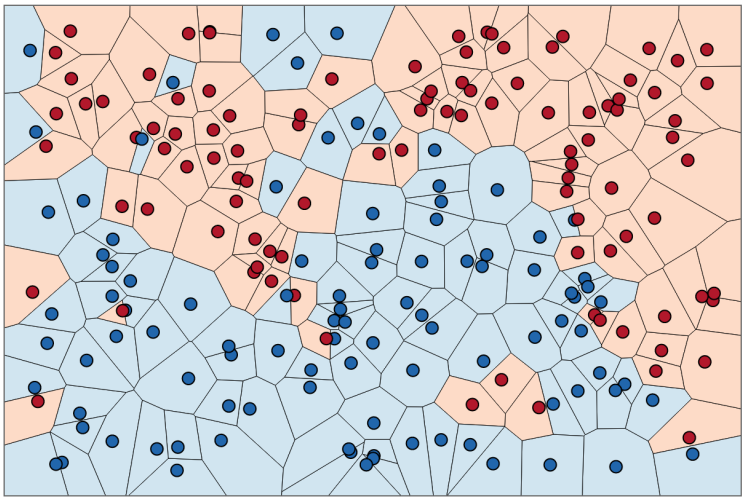


Figure 3: 1NN decision boundaries - Nearest neighborhoods for each point of the training data set.

Note that there will be zero error on training set (unless there are training data points that have identical feature values, but different labels).

K nearest neighbors

Instead of 1 closest sample, we find K: Let N_0 be the set of K training points that are closest to $\mathbf{x}_0$.

How do we use this set for (1) classification? (2) regression?



Figure 4: 3 nearest neighbors.

KNN for classification

Idea: Estimate conditional probability for a class as fraction of points among neighbors with the class label.

Remember: Let N_0 be the set of K training points that are closest to $\mathbf{x}_0$.

Then, we can estimate the per-class conditional probability given the sample x_0.

For each class $m \in M$:

$$P(y = m|\mathbf{x}_0) = \frac{1}{K} \sum_{(\mathbf{x}_i, y_i) \in N_0} I(y_i = m)$$

where $I(y_i = m)$ is 1 if $(\mathbf{x}_i, y_i) \in N_0$ is a member of class m, 0 otherwise.

- We can then select the class with the highest probability.
- Practically: select the most frequent class among the neighbors.

KNN for regression

Idea: Use the combined label of the K nearest neighbors. For example, we can take their mean:

$$\hat{y}_0 = \frac{1}{K} \sum_{(\mathbf{x}_i, y_i) \in N_0} y_i$$

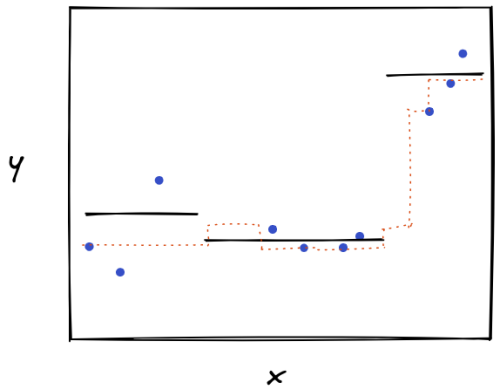


Figure 5: Example of a regression using median vote of the 3NN. The “true function” is shown in black, the orange dashed line shows the prediction of the regression model.

What value of K? Illustration (1NN)

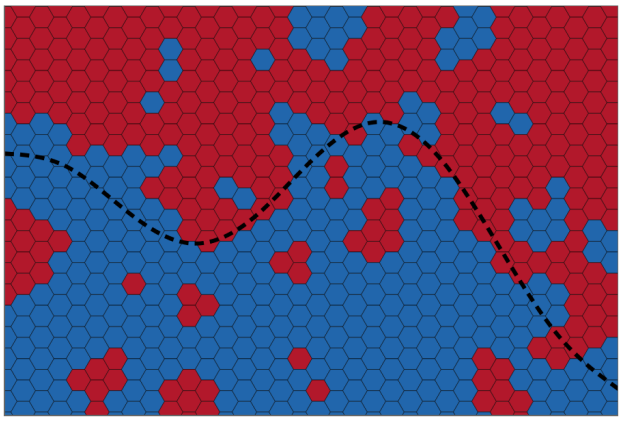


Figure 6: 1NN

What value of K? Illustration (2NN)

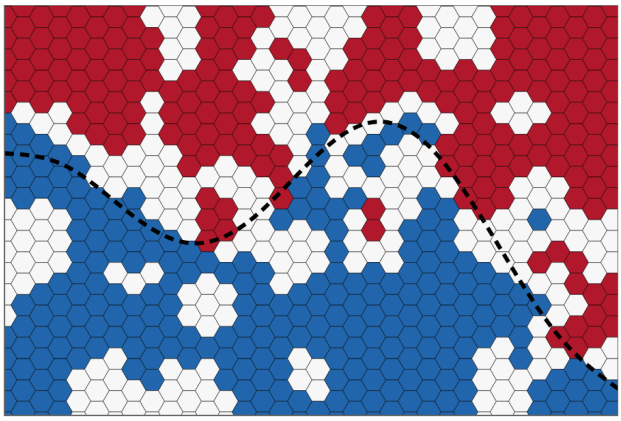


Figure 7: 2NN

What value of K? Illustration (3NN)

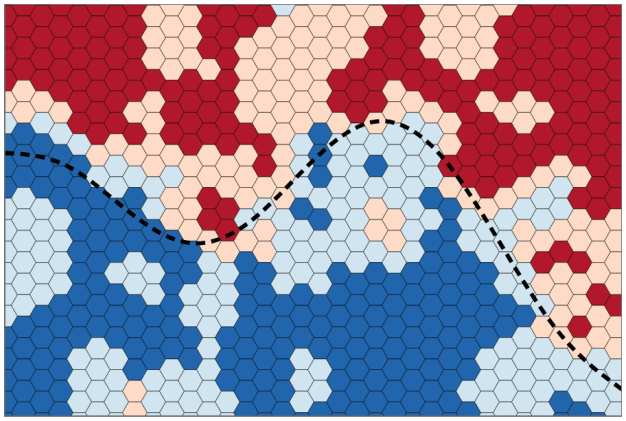


Figure 8: 3NN

What value of K? Illustration (9NN)

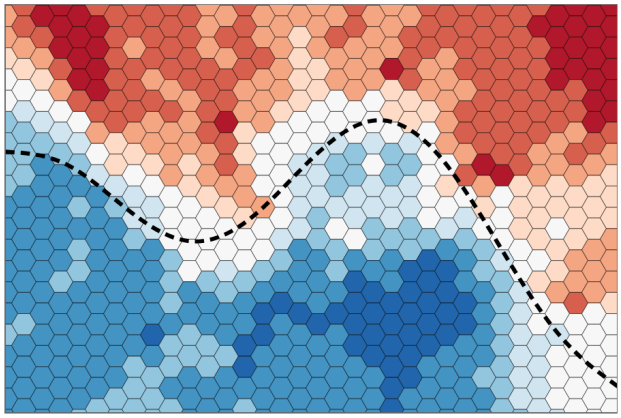


Figure 9: 9NN

What value of K? (1)

- In general: larger K, less complex model
- K can be selected by CV.
- Often cited “rule of thumb”: use $K = \sqrt{N}$

What value of K? (2)

- Alternative to fixed K: Radius-based neighbor learning.
- A fixed radius r is specified, can be selected by CV.
- Number of neighbors depends on local density of points.

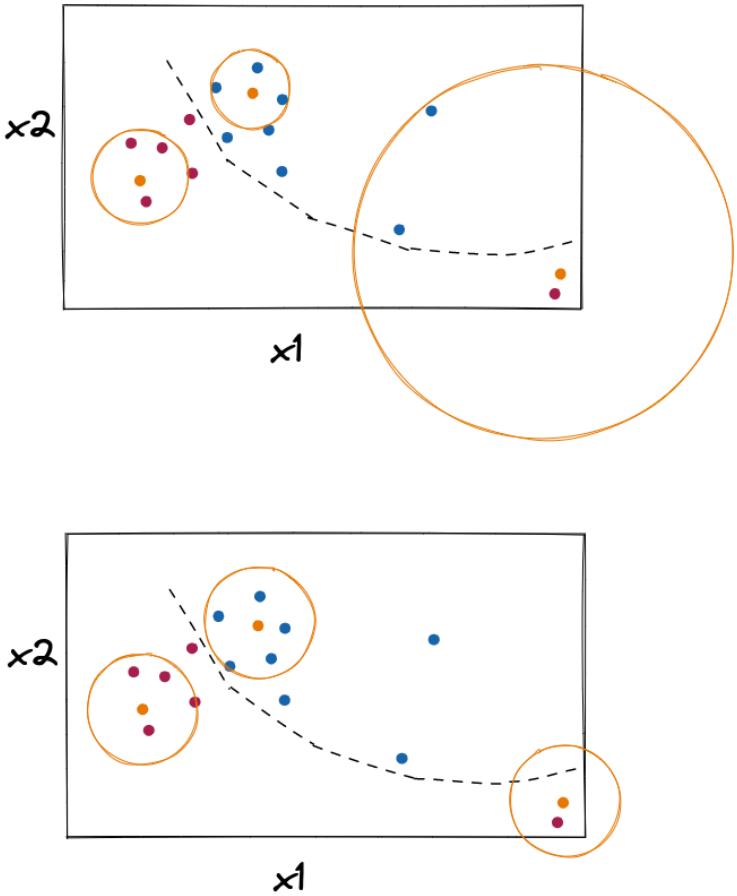


Figure 10: Comparison of KNN (top) and radius-based neighbor learning (bottom) when the density of training points in the feature space is uneven.

What distance measure? (1)

Some popular choices: for two vectors a_i, b_i,

- Euclidean (L2): $\sqrt{\sum_{i=1}^{d}(a_i - b_i)^2}$
- Manhattan (L1): $\sum_{i=1}^{d}|a_i - b_i|$

(L2 distance prefers many medium-sized disagreements to one big one.)

There are many more choices - for example, look at the distance metrics implemented in sklearn.

Problems with the basic distance metrics:

- When features have different scale/range, need to standardize
- KNN implicitly weights all features equally: this is a problem if you have features that are not relevant for the target variable!
- For images: pixel-wise distance doesn't necessarily equate to perceptual similarity

Distance measure - standardization (1)

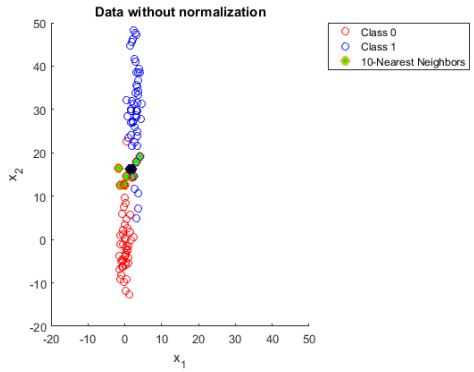


Figure 11: Without standardization, via https://stats.stackexchange.com/a/287439/. The x2 feature dominates the distance measure.

Distance measure - standardization (2)

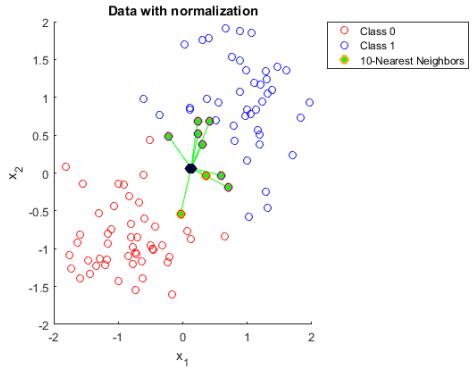


Figure 12: With standardization, via https://stats.stackexchange.com/a/287439/

Distance measure - equal weighted features

Suppose you are trying to predict a student's course grade using their previous GPA and how far they live from the NYU campus:

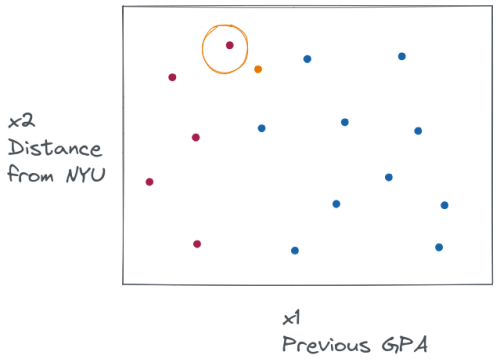


Figure 13: The training point circled in orange is the nearest neighbor, but it's not the most similar according to the only feature that matters (previous GPA). All features are weighted equally in the distance metric, regardless of their importance.

Alternative to equal weighted features: assign feature weights

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^{d} w_i (a_i - b_i)^2}$$

But then we need a way to assign feature weights!

Nearest neighbor runtime

- Training: just store data
- Inference: need to
  - compute distance to each of N points
  - distance metric typically scales with d

Predicting *each* test point requires a computation with that test point and *every* training sample. How does this compare to linear regression or logistic regression?

There, inference was much cheaper. We can tolerate a long training time more easily than a long inference time.

Summary of NN method

The good and the bad (1)

Good:

- Good interpretability
- Fast “learning” (*memory-based*)
- Works well in low dimensions for complex decision surfaces

The good and the bad (2)

Neutral:

- Assumes similar inputs have similar outputs

The good and the bad (3)

Bad:

- Slow prediction (especially with large N)

Flexible decisions with cheap prediction?

KNN was very flexible, but prediction is **slow**.

Next: flexible decisions, non-parametric approach, fast prediction

Idea: In KNN, we find the "neighborhood" of a test point and then give it the value of training points in that "neighborhood" - but it takes too long at inference time to define the "neighborhood".

What if we define "neighborhoods" and their values in advance, at training time? Then at inference time, we only need to determine which "neighborhood" a test point belongs in.

However, we run into another **computationally hard** problem! To partition the feature space into *optimal* neighborhoods is too expensive. Instead, we will rely on some heuristics and get a non-optimal, but good enough, partition.

Decision trees

Tree terminology

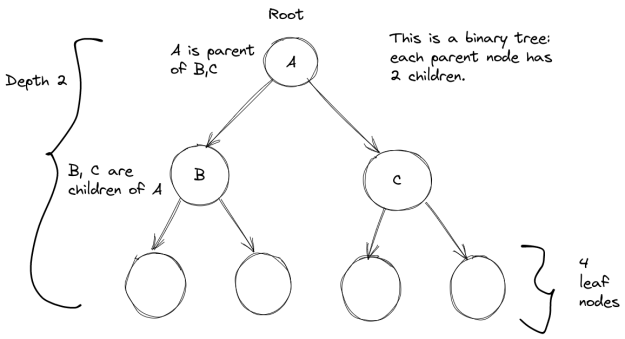


Figure 14: A binary tree.

- size of tree $|T|$ (number of leaf nodes)
- depth (max length from root node to a leaf node)

Note on notation

Following notation of ISL, Chapter 8:

- X_j is feature j
- x_i is sample i

Stratification of feature space (1)

Given set of possible predictors, $X_1, \dots, X_p$

- Training: Divide predictor space (set of possible values of X) into J non-overlapping regions: $R_1, \dots, R_J$, by splitting sequentially on one feature at a time.

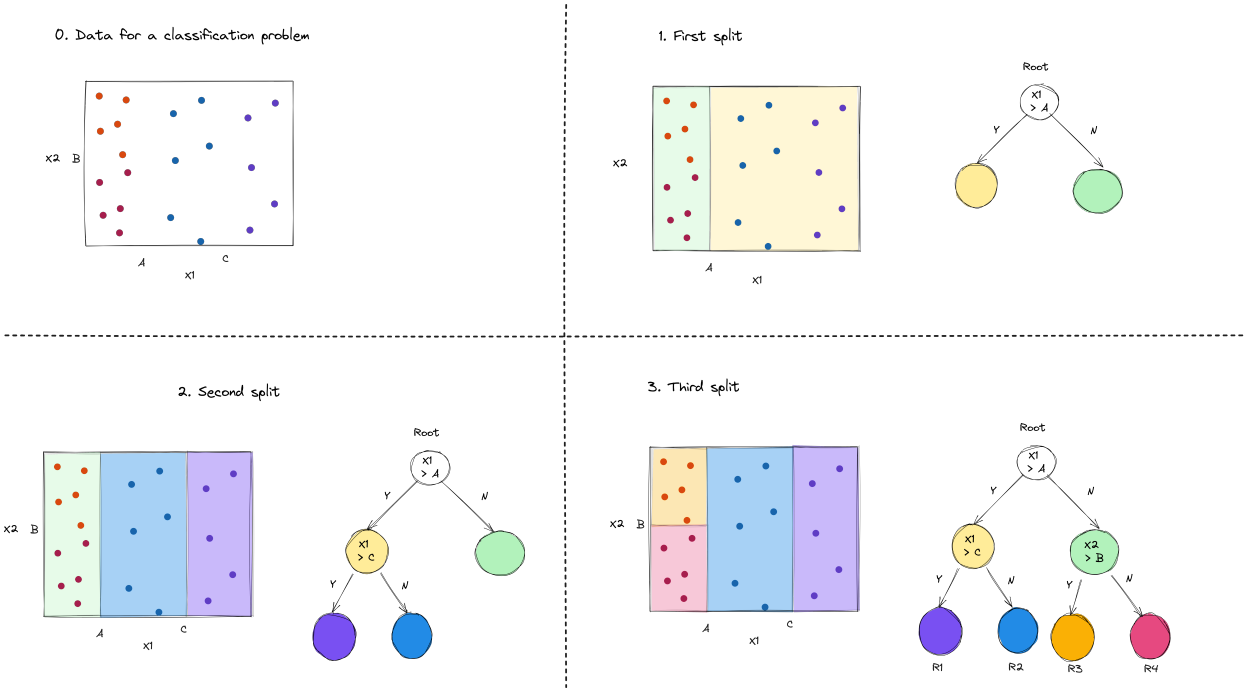


Figure 15: Dividing the feature space with a decision tree.

Stratification of feature space (2)

Prediction:

- At node that is not a leaf: test one feature X_i
- Branch from node depending on value of X_i
- At leaf node: for observation that falls in region R_j, predict $\hat{y}_{R_j}$
  - mean of labels of training points in R_j (regression)
  - mode of labels of training points in R_j (classification)

Stratification of feature space - illustration

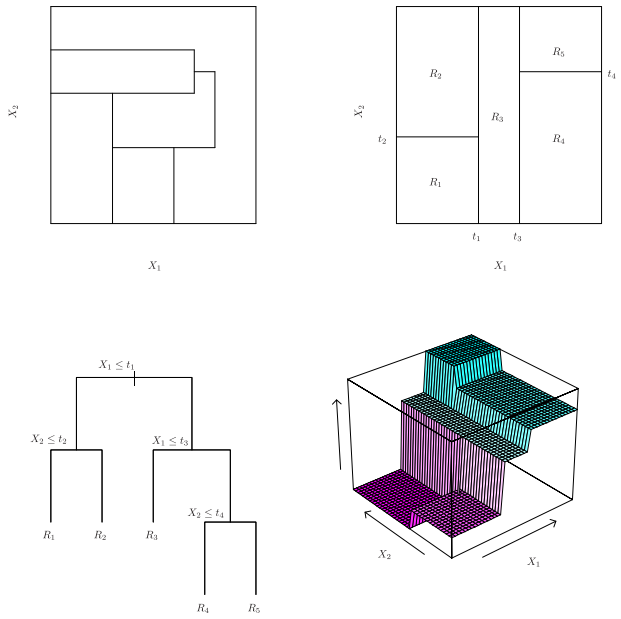


Figure 16: ISLR, Fig. 8.3.

The stratification on the top left cannot be produced by a decision tree using recursive binary splitting. The other three subfigures represent a single stratification. Note that the decision tree fits a piecewise step function!

Approximation using steps

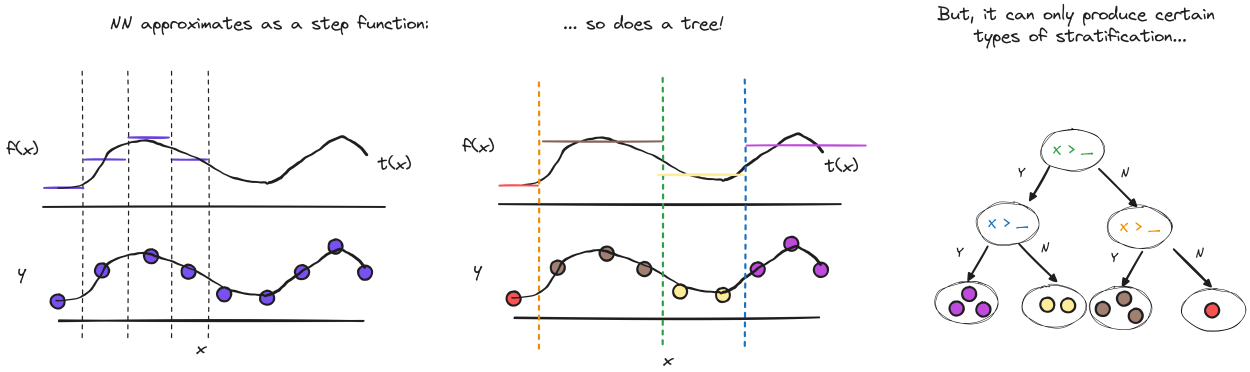


Figure 17: Like nearest neighbor, a tree represents an arbitrary function using steps.

Training a decision tree

Basic idea

- Goal: find the high-dimensional rectangles that minimize error
- Practical approach: recursive binary splitting (top-down, greedy approach)

It is too computationally expensive to consider every possible partition. Instead, we'll use a greedy algorithm: at each step, make the best "local" decision at that step, without looking ahead and making a decision that might yield better results at future steps

Recursive binary splitting steps

Start at root of the tree, considering all training samples.

1. At the current node,
2. Find feature X_j and *cutpoint* s that minimizes some *local* loss function (?)
3. Split training samples at that node into two leaf nodes
4. Stop when no training error (?)
5. Otherwise, repeat at leaf nodes

Recursive binary splitting

For any feature j and *cutpoint* s, define the regions

$$R_1(j,s) = \{X|X_j < s\}, \quad R_2(j,s) = \{X|X_j \geq s\}$$

where $\{X|X_j < s\}$ is the region of predictor space in which X_j takes on a value less than s.

Loss function for regression tree

For regression: look for feature j and cutpoint s that leads to the greatest possible reduction in squared error, where the "new" squared error is ($\hat{y}_{R_j}$ is the prediction for the samples in R_j):

$$\sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 \quad + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

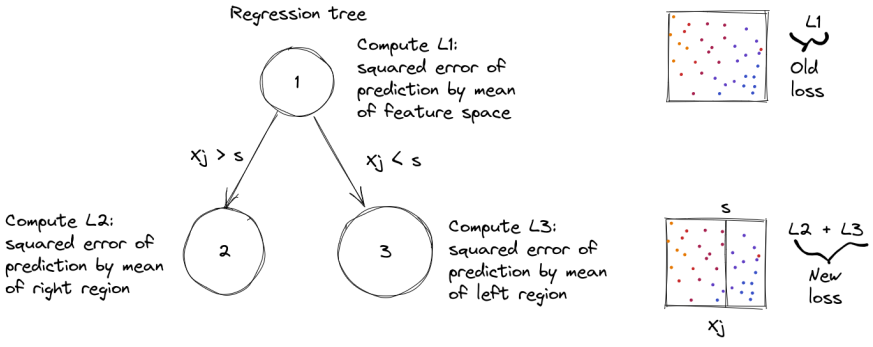


Figure 18: Training a regression tree.

Note: we are *not* computing the average loss of each region, we want the loss to be weighted by the number of samples it applies to!

Loss function for classification tree

For classification, find a split that minimizes some measure of node *impurity*:

- A node whose samples all belong to the same class - most *pure*
- A node whose samples are evenly distributed among all classes - highly *impure*

Measures of node impurity

Measure	Formula
Classification error rate (0-1 loss)	$\sum_{x_i \in R_m} 1(y_i \neq \hat{y}_{R_m})$
GINI index	$\sum_{k=0}^{K-1} \hat{p}_{mk}(1 - \hat{p}_{mk})$
Entropy	$-\sum_{k=0}^{K-1} \hat{p}_{mk} \log_2 \hat{p}_{mk}$

where $\hat{p}_{mk}$ is the proportion of training samples in R_m belonging to class k.

Classification error rate is not used often (this is a good explanation re: why), but it is used for *pruning*.

You can see that the GINI index is small when all values of $\hat{p}_{mk}$ are around 0 or 1.

Comparison - measures of node impurity

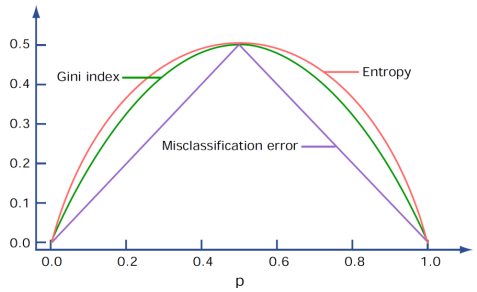


Figure 19: Measures of node “impurity”.

Conditional entropy

- Splitting on feature X creates subsets S_1 and S_2 with different entropies
- Conditional entropy:

$$\text{Entropy}(S|X) = \sum_v \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Information gain

- Choose feature to split so as to maximize information gain, the expected reduction in entropy due to splitting on X:

$$\text{Gain}(S, X) := \text{Entropy}(S) - \text{Entropy}(S|X)$$

Example: should I play tennis? (1)

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Figure 20: Via Tom Mitchell.

Example: should I play tennis? (2)

For top node: $S = \{9+, 5-\}, |S| = 14$

$$\text{Entropy}(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.94$$

Example: should I play tennis? (3)

If we split on Wind:

Considering the Weak branch:

- $S_{\text{weak}} = \{6+, 2-\}, |S_{\text{weak}}| = 8$
- $\text{Entropy}(S_{\text{weak}}) = -\frac{6}{8} \log_2(\frac{6}{8}) - \frac{2}{8} \log_2(\frac{2}{8}) = 0.81$

Considering the Strong branch:

- $S_{\text{strong}} = \{3+, 3-\}, |S_{\text{strong}}| = 6$
- $\text{Entropy}(S_{\text{strong}}) = 1$

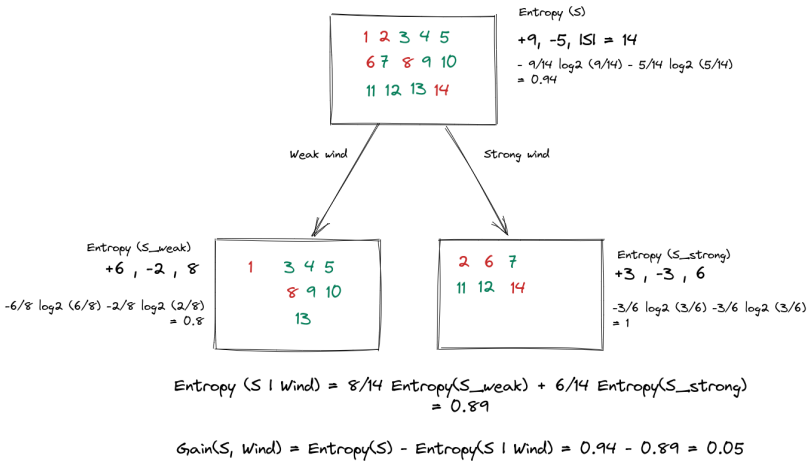


Figure 21: Considering the split on Wind.

Example: should I play tennis? (4)

$$\text{Entropy}(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

$$\text{Entropy}(S|\text{Wind}) = \frac{8}{14}\text{Entropy}(S_{\text{weak}}) + \frac{6}{14}\text{Entropy}(S_{\text{strong}}) = 0.89$$

$$\text{Gain}(S, \text{Wind}) = 0.94 - 0.89 = 0.05$$

Example: should I play tennis? (5)

- $\text{Gain}(S, \text{Outlook}) = 0.246$
- $\text{Gain}(S, \text{Humidity}) = 0.151$
- $\text{Gain}(S, \text{Wind}) = 0.048$
- $\text{Gain}(S, \text{Temperature}) = 0.029$

$\rightarrow$ Split on Outlook!

Variations

In this example, the data had only categorical variables, and no missing values.

What if we had a continuous (not categorical) variable? We would need to also decide how to partition the continous feature into a discrete set of intervals.

There are a few well-known algorithms for fitting decision trees - CART, ID3, C4.5 - that have different capabilities with respect to continuous features, features with missing values, and what measure of node impurity is used.

e.g. C4.5 introduces the idea that if a sample has a missing value for a feature,

- when training, compute information gain using only samples where the feature is defined
- when using, we decide which branch to follow based on which is more probable

Bias and variance

Managing tree depth

- If tree is too deep - likely to overfit (high variance)
- If tree is not deep enough - likely to have high bias

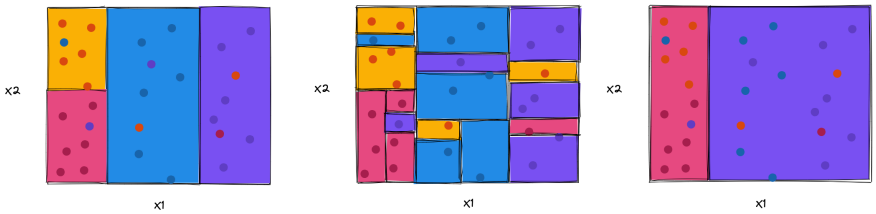


Figure 22: The depth/size of the tree (number of regions) controls the complexity of the regression line or decision boundaries, and the bias variance tradeoff.

Stopping criteria

If we build tree until there is zero error on training set, we have “memorized” training data.

Other stopping criteria:

- Max depth
- Max size (number of leaf nodes)
- Min number of samples to split
- Min number of samples in leaf node
- Min decrease in loss function due to split

(Can select depth, etc. by CV)

Pruning

- Alternative to stopping criteria: build entire tree, then *prune*
- With greedy algorithm - a very good split may descend from a less-good split

Weakest link pruning (1)

Prune a large tree from leaves to root:

- Start with full tree T_0
- Merge two adjacent leaf nodes into their parent to obtain T_1 by minimizing:

$$\frac{Err(T_1) - Err(T_0)}{|T_0| - |T_1|}$$

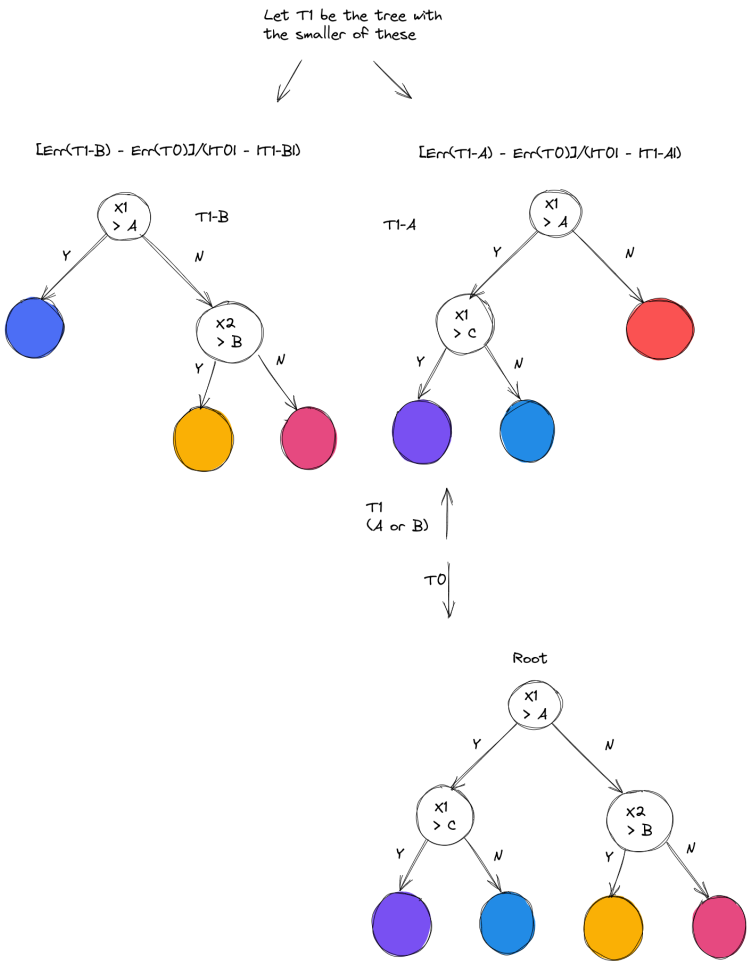


Figure 23: Weakest link pruning.

We usually prune classification trees using classification error rate as loss function, even if tree was built using GINI or entropy.

Weakest link pruning (2)

- Iterate to produce a sequence of trees $T_0, T_1, \ldots, T_m$ where T_m is a tree of minimum size.
- Select optimal tree by CV

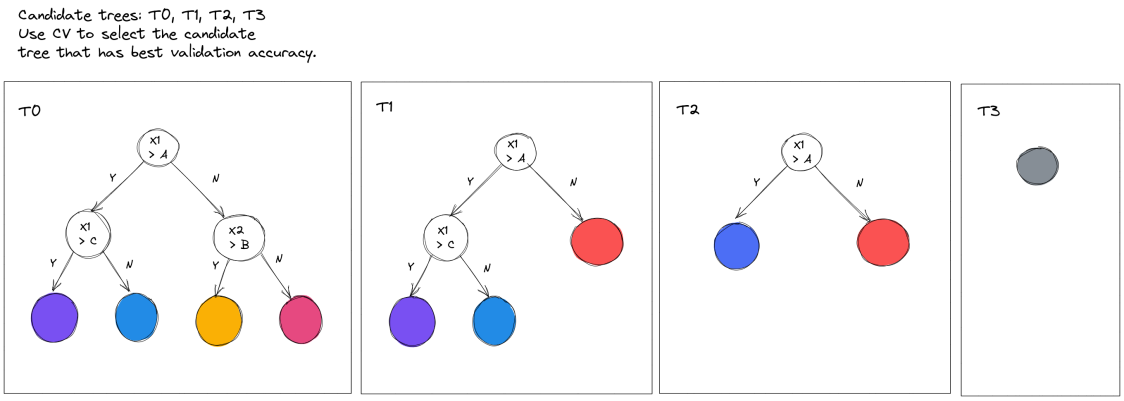


Figure 24: Selecting tree from the set of candidate trees.

Cost complexity pruning

Equivalent to: Minimize

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

Choose α by CV, 1-SE rule ($\uparrow \alpha, \downarrow |T|$).

Runtime

Inference time

We have to make d comparisons, where d is depth of tree...

(In a well-balanced tree, we halve the number of samples in each split.)

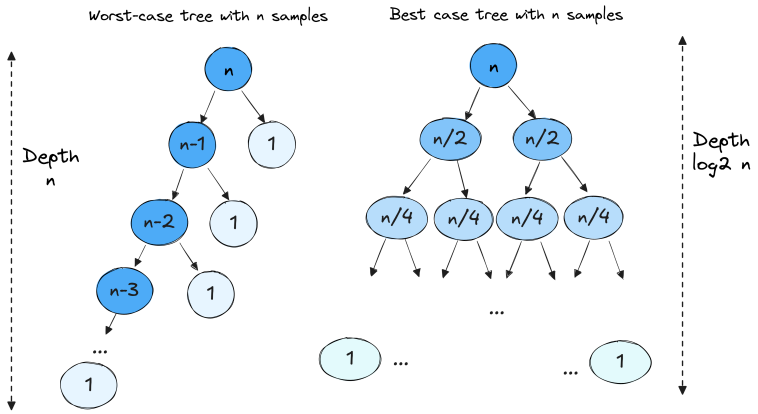


Figure 25: Tree depth in worst-case and well-balanced trees.

- Worst case: $O(N)$
- Well-balanced: $O(\log N)$

Summary - so far

The good and the bad (1)

Good:

- Flexible with much faster inference time than KNN
- Easy to interpret, close to human decision-making
- Can derive feature importance
- Easily handles mixed types, different ranges

The good and the bad (2)

Bad:

- Need greedy heuristic to train
- Deep trees have large variance
- Non-robust: Small change in data can cause large change in estimated tree

Ensembles

Recap: decision trees

- Let trees grow deep - low bias, high variance
- Don't let trees get deep: low variance, high bias

Ensemble methods - the idea

Combine multiple **weak learners** - having either high bias or high variance - to create an **ensemble** with better prediction

Ensemble methods - types (1)

- Combine multiple learners with high **variance** in a way that reduces their variance
- Combine multiple learners with high **bias** in a way that reduces their bias

Ensemble methods - types (2)

- **Parallel**: build base estimators *independently* and then average their predictions. Combined estimator is usually better than any single base estimator because its *variance* is reduced.
- **Sequential**: (boosting) build base estimators *sequentially* and each one tries to reduce the *bias* of the combined estimator.

Bagging

Bagging - background

- Designed for, and most often applied to, decision trees
- Name comes from **bootstrap aggregation**

Bootstrapping

- Basic idea: Sampling **with replacement**
- Each "bootstrap training set" is *same size* as full training set, and is created by sampling with replacement
- Some samples will appear more than once, some samples not at all

Bootstrap aggregation

- Create multiple versions $1, \dots, B$ of training set with bootstrap
- Independently train a model on each bootstrap training set.
- For regression, average the predictions:

$$\hat{y}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x).$$

- For classification, use the majority vote as the final class:

$$\hat{y}(x) = \text{mode}_{b=1,\dots,B}\, \hat{y}_b(x).$$

Bagging trees

- Construct B trees using B bootstrapped training sets
- Let the trees grow deep, no pruning
- Each individual tree has low bias, high variance
- Average regression predictions or vote on class predictions to reduce variance (if independent!)

Correlated trees

Problem: trees produced by bagging are highly correlated, and so we don't get much benefit due to averaging them.

- Imagine there is one feature that is strong predictor, several moderate predictors
- Most/all trees will split on this feature
- Averaging correlated quantities does not reduce variance as much.

Random forests

Grow many decorrelated trees:

- **Bootstrap**: grow each tree with bootstrap resampled data set.
- **Split-variable randomization**: Force each split to consider *only* a subset of m of the p predictors.

e.g. $m = \frac{p}{3}$, but this should be considered a tuning parameter.

Bagged trees illustration

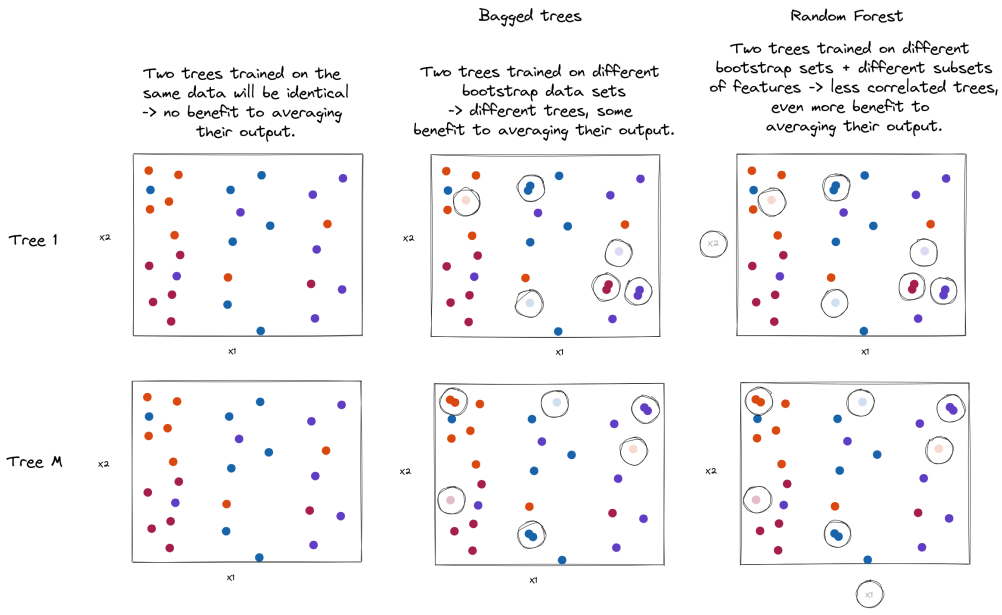


Figure 26: Identical data, bootstrapped data, and bootstrapped data with split variable randomization.

A note on computation

- Bagged trees and random forests can be fitted in parallel on many cores!
- Each tree is built independently of the others

Boosting

Boosting - training

Iteratively build a succession of models:

- Train a weak model. Typically a very shallow tree.
- In training set for bth model, focus on errors made by $b-1$th model.
- Use (weighted) model output
- Reduces bias *and* variance!

AdaBoost (Adaptive Boosting)

Adjust *weights* so that each successive model focuses on more "difficult" samples.

Consider a binary classification problem, where $y_i \in \{-1, +1\} \forall i$.

AdaBoost algorithm

1. Let $w_i = \frac{1}{N}$ for all i in training set.
2. For $m = 1, \dots, M$, repeat:

AdaBoost algorithm (inner loop)

- Fit weak learner $\hat{f}^m$, on training data with sample weights w_i.
- Compute weighted error err_m:

$$err_m = \frac{\sum_{i=1}^{N} w_i 1(y_i \neq \hat{f}^m(x_i))}{\sum_{i=1}^{N} w_i}$$

- Compute coefficient $\alpha_m = \log\left(\frac{1-err_m}{err_m}\right)$
- Update weights: $w_i \leftarrow w_i e^{\alpha_m 1(y_i \neq \hat{f}^m(x_i))}$ (for misclassified samples, scale weight by e^{α_m})

AdaBoost algorithm (final step)

3. Output final ensemble model:

$$f_M(x) = \sum_{m=1}^{M} \alpha_m \hat{f}^m(x), \quad \hat{y}(x) = \text{sign}[f_M(x)]$$

Gradient Boosting

- General goal of boosting: find the model at each stage that minimizes loss function on ensemble (computationally difficult!)
- AdaBoost interpretation (discovered years later): Gradient descent algorithm that minimizes exponential loss function.
- Gradient boosting: works for any differentiable loss function. At each stage, find the local gradient of loss function, and take steps in direction of steepest descent.

Recap

Summary of (selected) ensemble methods

- Can use a single estimator that has poor performance
- Combining the output of multiple estimators into a single prediction: better predictive accuracy, less interpretability
- Also more expensive to fit